DAY 2 · AWS LINUX LAB

Storage & Filesystems

EBS volumes & LVM

Attached a second EBS volume, formatted it, mounted it permanently via `/etc/fstab`, and grew it live from 5 GB to 6 GB.

Then built a full LVM stack from scratch on a third volume — Physical Volume, Volume Group, Logical Volume — grew it live, and cleanly tore it all down afterward.

Local — your machine · Remote — the server · Config file · Concept · AWS Console · Gotcha · Best practice · Paths

What we covered

- ✓ Created and attached a second EBS volume, matching the instance's Availability Zone
- ✓ AWS device names (`/dev/sdd`) vs. the actual Linux/NVMe names (`/dev/nvme1n1`)
- ✓ Formatted with ext4, mounted manually, then made it permanent via `/etc/fstab` using the UUID
- ✓ Verified the fstab entry safely with `umount` + `mount -a` before trusting a reboot
- ✓ Grew the volume in the AWS Console (5 GB → 6 GB) and live-resized the filesystem to match
- ✓ Built a full LVM stack on a third volume: PV → VG → LV
- ✓ Hit and resolved a real extent-rounding error when extending an LV
- ✓ Live-extended the LV using all remaining free space in the pool
- ✓ Cleanly tore down the LVM stack and removed the test volume

Part 1 — EBS volume, ext4 & fstab

AWS Console Create & attach the volume

Key rule · The volume's Availability Zone must exactly match the instance's AZ (e.g. `eu-north-1b`) — otherwise it can't be attached.

Remote Identify the new disk

```
lsblk   # list all block devices
```

```
nvme1n1   259:5    0    5G   0 disk   # the new, empty volume — AWS called it /dev/sdd
```

Remote Format & mount

```
sudo mkfs.ext4 /dev/nvme1n1         # create an ext4 filesystem on the raw disk
sudo mkdir -p /mnt/data             # -p: create the mountpoint
sudo mount /dev/nvme1n1 /mnt/data   # mount it
df -h /mnt/data                     # -h: confirm it's mounted, human sizes
```

Remote Make the mount permanent

```
sudo blkid /dev/nvme1n1   # get the filesystem's UUID
sudo nano /etc/fstab      # add the mount line
```

```
/etc/fstab
UUID=f707c3d8-b4d8-4932-b350-4776695c932d  /mnt/data  ext4  defaults  0  2
```

UUID=...	/mnt/data	ext4	defaults	0	2
which filesystem	where to mount	filesystem type	standard options	dump: skip backup	fsck order (after root)

Gotcha · fstab only documents comments as lines that start with `#` — keep notes on their own line, never after the fields. The field table above does the explaining instead.

```
sudo umount /mnt/data   # unmount (note: "umount", no "n")
sudo mount -a           # -a: mount all fstab entries, like a boot
df -h /mnt/data         # confirm it came back via fstab
```

Concept Why UUID, not device name

Concept · Device names like `/dev/nvme1n1` can shift when disks are added or removed. A filesystem's UUID is permanent and unique, so fstab always finds the right disk regardless of the order disks are detected in.

Remote Live resize after growing the volume in AWS

```
lsblk                       # confirm the kernel sees the new size (6G)
sudo resize2fs /dev/nvme1n1 # grow the filesystem to fill the disk
df -h /mnt/data             # confirm ~6G, no unmount needed
```

Part 2 — Full LVM stack

Concept The LVM stack

Disk → Physical Volume (PV) → Volume Group (VG) → Logical Volume (LV) → Filesystem → Mount

Remote Build PV → VG → LV on a third volume

```
sudo pvcreate /dev/nvme2n1              # mark the disk as LVM-managed
sudo vgcreate vg-data /dev/nvme2n1      # create a pool from the PV
sudo lvcreate -L 3G -n lv-data vg-data  # -L: size, -n: name of the new LV
```

Remote Format & mount the LV

```
sudo mkfs.ext4 /dev/vg-data/lv-data            # ext4 filesystem on the LV
sudo mkdir -p /mnt/lvm-data                    # -p: create the mountpoint
sudo mount /dev/vg-data/lv-data /mnt/lvm-data  # mount it
df -h /mnt/lvm-data                            # confirm size and mount
```

Bug · extent rounding on lvextend

CAUSE `lvextend -L +1G` failed — "256 extents needed, but only 255 available." VG metadata takes a few MiB, so the pool had slightly less than a clean 1 GiB free.

FIX Use `-l +100%FREE` to take all remaining space instead of an exact size.

Remote Live-extend the LV

```
sudo lvextend -l +100%FREE /dev/vg-data/lv-data  # -l: grow by all free space in the pool
sudo resize2fs /dev/vg-data/lv-data              # grow the filesystem to match
df -h /mnt/lvm-data                              # confirm ~3.9G, still mounted throughout
```

Remote Tear down the LVM stack

```
sudo umount /mnt/lvm-data           # unmount first
sudo lvremove /dev/vg-data/lv-data  # remove the LV (confirm with y)
sudo vgremove vg-data               # remove the volume group
sudo pvremove /dev/nvme2n1          # release the disk from LVM
sudo rmdir /mnt/lvm-data            # remove the empty mountpoint
```

AWS Console Remove the test volume

Cleanup · Detach the 4 GiB LVM test volume once it shows **Available**, then delete it. The 6 GiB `/mnt/data` volume stays — it's real, mounted infrastructure, not a test.

Where things live

What	Path
Permanent mount config	`/etc/fstab`
Direct-disk mountpoint	`/mnt/data`
LVM logical volume path	`/dev/<vg-name>/<lv-name>`
LVM device-mapper path	`/dev/mapper/<vg>-<lv>`

Best practices

Match the AZ
The EBS volume's AZ must match the instance's AZ before attaching.

UUID in fstab
Not device names — those can shift.

Test before rebooting
Check a new fstab entry with `umount` + `mount -a` before trusting a real reboot.

Grow in two steps
An EBS volume grows live in AWS, but the filesystem must be resized separately to see it.

LVM resizes live
Resize storage without unmounting — a real advantage over a raw disk.

Use +100%FREE
Prefer `-l +100%FREE` over an exact `-L` size to avoid extent-rounding failures.

Tear down in order
LV → VG → PV, before removing the underlying disk.

Name LVM parts clearly
Prefixes like `vg-` and `lv-` make the stack readable at a glance.

Part of the AWS/Linux sysadmin lab · see day1-naming-conventions for the naming reference